

# Bisimulation sémantique entre un DSL déclaratif de planification et MiniZinc

Notes de Recherche - Master 2

7 mai 2026

## Résumé

Ce document formalise la correspondance sémantique entre un langage dédié (DSL) de planification déclaratif et son encodage en MiniZinc. L'objectif est d'établir une *bisimulation sémantique* prouvant que la traduction préserve de manière exacte l'espace des solutions, les contraintes et l'optimalité liée aux préférences. Nous y introduisons les concepts de correction, de complétude et de préservation de l'optimalité à travers des fonctions d'encodage et de décodage. Une application concrète sur la planification de soutenances académiques illustre cette démarche, garantissant ainsi l'intégrité de bout en bout de la chaîne de résolution.

## Table des matières

|           |                                                                            |          |
|-----------|----------------------------------------------------------------------------|----------|
| <b>1</b>  | <b>Introduction</b>                                                        | <b>2</b> |
| <b>2</b>  | <b>Motivation et Risques</b>                                               | <b>2</b> |
| <b>3</b>  | <b>Adaptation du Concept de Bisimulation</b>                               | <b>2</b> |
| <b>4</b>  | <b>Correspondance Sémantique DSL <math>\leftrightarrow</math> MiniZinc</b> | <b>3</b> |
| 4.1       | Définition Abstraite du DSL . . . . .                                      | 3        |
| 4.2       | Définition Abstraite du Modèle MiniZinc . . . . .                          | 3        |
| 4.3       | Fonctions de Traduction, d'Encodage et de Décodage . . . . .               | 3        |
| 4.4       | La Relation de Bisimulation $\mathcal{R}$ . . . . .                        | 4        |
| <b>5</b>  | <b>Théorèmes Fondamentaux</b>                                              | <b>4</b> |
| <b>6</b>  | <b>Exemple Concret : Planification de Soutenances</b>                      | <b>4</b> |
| <b>7</b>  | <b>Tableau de Correspondance Sémantique</b>                                | <b>5</b> |
| <b>8</b>  | <b>Méthodologie de Construction</b>                                        | <b>5</b> |
| <b>9</b>  | <b>Architecture et Chaîne de Résolution</b>                                | <b>6</b> |
| <b>10</b> | <b>Discussion et Limites</b>                                               | <b>6</b> |
| <b>11</b> | <b>Conclusion</b>                                                          | <b>6</b> |

# 1 Introduction

Dans le cadre de la conception d'une plateforme de planification déclarative, nous avons développé un DSL (Domain-Specific Language) permettant à un utilisateur de modéliser des problèmes de planification via des fichiers `.planning`. Ces fichiers décrivent des activités, des ressources, des rôles, ainsi que des contraintes et des éventuelles préférences. Pour résoudre ces problèmes, notre approche repose sur la traduction automatique de ce DSL vers le langage MiniZinc, qui sert de pivot avant l'aplatissement en FlatZinc et la résolution par des solveurs spécialisés (Gecode, Chuffed, OR-Tools CP-SAT, HiGHS, COIN-BC).

La validité de cette plateforme repose entièrement sur la fidélité de cette traduction. Si le modèle MiniZinc généré ne correspond pas rigoureusement au problème exprimé dans le DSL, les solutions retournées par les solveurs seront erronées ou sous-optimales. Ce rapport a pour objectif de formaliser cette correspondance à travers le concept de **bisimulation sémantique**.

## 2 Motivation et Risques

Prouver formellement que la sémantique du DSL est préservée lors de sa traduction en MiniZinc est une étape cruciale de notre démarche de recherche. L'absence d'une telle garantie nous expose à plusieurs risques majeurs :

- **Fausse solutions (Soundness failure)** : Le solveur retourne une solution qui satisfait le modèle MiniZinc, mais qui viole une contrainte métier implicite ou explicite du DSL.
- **Perte de solutions (Completeness failure)** : Un planning parfaitement valide selon le DSL est rejeté par le solveur à cause d'une traduction excessivement restrictive en MiniZinc.
- **Mauvaise traduction des contraintes** : La logique des règles métier (précédences, exclusivités) est corrompue lors de la traduction.
- **Mauvaise traduction des préférences** : Les poids des pénalités sont mal mappés, conduisant à une **solution optimale MiniZinc qui ne correspond pas à une solution optimale côté DSL**.
- **Incohérence métier** : Une déconnexion totale entre le problème réel (ex. : l'organisation de soutenances) et l'abstraction mathématique résolue par la machine.

## 3 Adaptation du Concept de Bisimulation

Historiquement, la notion de bisimulation, introduite par Milner et formalisée par Sangiorgi [2, 3], s'applique aux systèmes de transitions d'états (ex. : processus concurrents). Elle stipule que deux systèmes sont bisimilaires si chaque étape de l'un peut être reproduite par l'autre, et vice-versa.

Dans notre contexte, le DSL et MiniZinc ne sont pas des langages impératifs avec des états transitionnels, mais des **langages déclaratifs**. L'exécution ne consiste pas à changer d'état, mais à caractériser un **espace de solutions**. Nous adaptons donc le terme de "bisimulation" pour désigner une **correspondance sémantique bidirectionnelle et statique** entre les deux univers :

- **L’univers DSL** : L’espace des plannings valides (affectations d’activités à des créneaux, salles et ressources selon des rôles).
- **L’univers MiniZinc** : L’espace des valuations des variables décisionnelles qui satisfont l’ensemble des contraintes du modèle généré.

La bisimulation signifie ici qu’il existe un isomorphisme (ou du moins une bijection décodable) entre ces deux espaces de solutions, préservant non seulement la faisabilité mais aussi l’ordre total induit par les pénalités d’optimisation. Cette nuance est fondamentale pour une utilisation correcte du terme dans notre mémoire de recherche.

## 4 Correspondance Sémantique DSL $\leftrightarrow$ MiniZinc

### 4.1 Définition Abstraite du DSL

Soit  $\mathcal{P}_{DSL} = (\mathcal{V}_{DSL}, \mathcal{C}_{DSL}, \mathcal{O}_{DSL})$  un problème exprimé dans notre DSL.

- $\mathcal{V}_{DSL}$  représente l’ensemble des entités du domaine : domaines, activités, ressources, rôles, créneaux, salles.
- $\mathcal{C}_{DSL}$  est l’ensemble des contraintes dures.
- $\mathcal{O}_{DSL}$  est l’ensemble des préférences souples.

Une **Solution DSL** est un planning valide, c’est-à-dire une affectation  $\sigma$  où pour chaque activité  $a \in \mathcal{A}$ , on associe un créneau, une salle, et un ensemble de ressources tenant les rôles requis, tout en satisfaisant  $\mathcal{C}_{DSL}$ . L’ensemble de ces solutions est noté  $Sol(\mathcal{P}_{DSL})$ .

Le coût d’une solution, noté  $Cost_{DSL}(\sigma)$ , est la somme des pénalités issues de la violation des préférences dans  $\mathcal{O}_{DSL}$ . Si  $\mathcal{O}_{DSL} = \emptyset$  (bloc `preferences` vide), il s’agit d’une **instance de satisfaction** ( $Cost_{DSL}(\sigma) = 0$ ). S’il est non vide, c’est une **instance d’optimisation**.

### 4.2 Définition Abstraite du Modèle MiniZinc

La génération automatique produit un modèle abstrait  $\mathcal{P}_{MZN} = (\mathcal{X}, \mathcal{D}, \mathcal{C}_{MZN}, obj)$ .

- $\mathcal{X}$  est l’ensemble des variables décisionnelles (tableaux d’entiers).
- $\mathcal{D}$  représente les domaines admissibles de ces variables.
- $\mathcal{C}_{MZN}$  est l’ensemble des contraintes MiniZinc.
- $obj$  est la fonction objectif.

Une **Valuation MiniZinc** est une affectation  $v : \mathcal{X} \rightarrow \mathbb{Z}$  respectant  $\mathcal{D}$  et satisfaisant  $\mathcal{C}_{MZN}$ . L’ensemble de ces valuations est noté  $Val(\mathcal{P}_{MZN})$ . Le coût associé est  $Cost_{MZN}(v)$  (la valeur de la variable `penalty`).

### 4.3 Fonctions de Traduction, d’Encodage et de Décodage

Nous définissons formellement :

- **Traduction** :  $\mathcal{T} : \mathcal{P}_{DSL} \rightarrow \mathcal{P}_{MZN}$ , l’algorithme qui génère le code MiniZinc à partir du fichier `.planning`.
- **Encodage** :  $encode : Sol(\mathcal{P}_{DSL}) \rightarrow Val(\mathcal{P}_{MZN})$ , la transformation théorique d’un planning valide en affectation d’entiers.
- **Décodage** :  $decode : Val(\mathcal{P}_{MZN}) \rightarrow Sol(\mathcal{P}_{DSL})$ , la fonction inverse utilisée par notre plateforme pour reconstruire un planning métier depuis le retour du solveur.

## 4.4 La Relation de Bisimulation $\mathcal{R}$

Nous définissons la relation de bisimulation  $\mathcal{R} \subseteq \text{Sol}(\mathcal{P}_{DSL}) \times \text{Val}(\mathcal{P}_{MZN})$  telle que  $\sigma \mathcal{R} v$  si et seulement si :

$$\text{encode}(\sigma) = v \quad \text{et} \quad \text{decode}(v) = \sigma$$

## 5 Théorèmes Fondamentaux

**Théorème 5.1** (Correction / Soundness). *Pour toute valuation  $v \in \text{Val}(\mathcal{P}_{MZN})$  retournée par la résolution de  $\mathcal{T}(\mathcal{P}_{DSL})$ , la solution décodée  $\sigma = \text{decode}(v)$  est un planning valide, i.e.,  $\sigma \in \text{Sol}(\mathcal{P}_{DSL})$ .*

*Intuition.* Chaque contrainte  $c \in \mathcal{C}_{DSL}$  est traduite de manière conservatrice en  $\mathcal{C}_{MZN}$ . Si  $v$  satisfait  $\mathcal{C}_{MZN}$ , la sémantique de  $\text{decode}$  garantit structurellement le respect de  $c$  dans l'espace DSL.  $\square$

**Théorème 5.2** (Complétude / Completeness). *Pour tout planning valide  $\sigma \in \text{Sol}(\mathcal{P}_{DSL})$ , il existe une valuation correspondante  $v \in \text{Val}(\mathcal{P}_{MZN})$  telle que  $\text{encode}(\sigma) = v$ .*

**Théorème 5.3** (Préservation du Coût et de l'Optimalité). *Pour toute solution  $\sigma \in \text{Sol}(\mathcal{P}_{DSL})$  et sa valuation bisimilaire  $v = \text{encode}(\sigma)$ , on a :*

$$\text{Cost}_{DSL}(\sigma) = \text{Cost}_{MZN}(v)$$

*Corollaire :* Si  $v^*$  est une valuation optimale minimisant la pénalité en MiniZinc, alors  $\text{decode}(v^*)$  est un planning optimal dans le DSL.

**Remarque 5.1.** Cette dualité gouverne la fin de la génération du modèle  $\mathcal{T}$  :

- **Satisfaction** : Si  $\mathcal{O}_{DSL} = \emptyset$ , le fichier se termine par *solve satisfy*.
- **Optimisation** : Si  $\mathcal{O}_{DSL} \neq \emptyset$ , les préférences sont converties en pénalités et le fichier se termine par *solve minimize penalty*.

## 6 Exemple Concret : Planification de Soutenances

Appliquons ce formalisme au cas principal de notre mémoire : la planification de soutenances.

- **Activité** :  $a$  correspond à une instance de "Soutenance".
- **Variables DSL** : salle, créneau, rôles (président, rapporteur, membre).
- **Variables MiniZinc générées** : `room[a]`, `slot[a]`, `president[a]`, `reviewer[a]`, `member[a]`.

Supposons que le solveur retourne la valuation  $v$  suivante :

```
slot[a] = 3
room[a] = 2
president[a] = 5
reviewer[a] = 7
member[a] = 9
```

La fonction de décodage  $\text{decode}(v)$  utilise les tables de symboles générées par la plateforme pour instancier la solution DSL  $\sigma$  :

- `slot 3`  $\implies$  "Mardi 10h00"
- `room 2`  $\implies$  "Salle de Conférence A"
- `president 5`  $\implies$  "Prof. Alan Turing"
- `reviewer 7`  $\implies$  "Prof. John von Neumann"
- `member 9`  $\implies$  "Dr. Grace Hopper"

La solution côté DSL s'écrit donc :

**Soutenance** *a* :

Créneau : Mardi 10h00

Salle : Salle de Conférence A

Jury : Alan Turing (Président), John von Neumann (Rapporteur), Grace Hopper (Membre).

Si cette valuation satisfait toutes les contraintes MiniZinc, la preuve de correction garantit que les règles métier (ex. Turing n'est pas déjà occupé ailleurs à 10h00) sont respectées dans ce planning.

## 7 Tableau de Correspondance Sémantique

| Concepts du DSL                                      | Équivalents MiniZinc                                                |
|------------------------------------------------------|---------------------------------------------------------------------|
| Espace de solutions valides $Sol(\mathcal{P}_{DSL})$ | Espace de valuations $Val(\mathcal{P}_{MZN})$                       |
| Entité (ex : Soutenance <i>a</i> )                   | Index dans un tableau (ex : $i \in 1..N$ )                          |
| Domaines (Rôles, Salles, Créneaux)                   | Déclaration <code>var 1..MAX</code> ou <code>var SET of 1..R</code> |
| Contraintes dures                                    | Prédicats booléens dans les blocs <code>constraint</code>           |
| Préférences souples                                  | Poids affectés aux violations et cumulés                            |
| Satisfaction (sans préférences)                      | Instruction finale <code>solve satisfy</code>                       |
| Optimisation (avec préférences)                      | Instruction finale <code>solve minimize penalty</code>              |

TABLE 1 – Traduction des concepts abstraits entre les deux univers

## 8 Méthodologie de Construction

Pour prouver cette bisimulation concrètement, nous avons adopté une méthodologie par étapes inspirée de l'ingénierie des compilateurs certifiés [4, 5] :

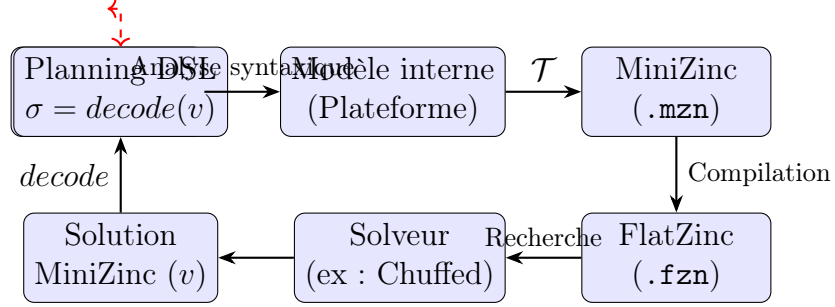
1. **Syntaxe abstraite du DSL** : Formalisation mathématique du fichier `.planning`.
2. **Sémantique du DSL** : Définition formelle d'un planning valide ensembliste.
3. **Sémantique de MiniZinc** : Axiomatisation du modèle généré.
4. **Traduction  $\mathcal{T}$**  : Algorithmes récursifs de génération de code.
5. **Définition de *encode* et *decode*** : Mise en place des dictionnaires d'index.
6. **Relation  $\mathcal{R}$**  : Formalisation du mapping bidirectionnel.
7. **Preuve de correction** : Démonstration analytique que le décodage préserve les contraintes.
8. **Preuve de complétude** : Vérification qu'aucune sur-contrainte n'a été insérée.

9. **Preuve d'optimisation** : Vérification de la linéarité des pénalités (`minimize penalty`).
10. **Validation expérimentale** : Tests automatisés de bout en bout avec Gecode et Chuffed.

## 9 Architecture et Chaîne de Résolution

Le schéma suivant illustre le cheminement d'un problème dans notre architecture, soulignant où la relation de bisimulation opère :

**Bisimulation Sémantique  $\mathcal{R}$**



## 10 Discussion et Limites

Bien que notre cadre théorique garantisse une équivalence stricte, son application pratique rencontre certaines limites. Tout d'abord, nous supposons implicitement la fiabilité du compilateur de la chaîne MiniZinc vers FlatZinc [6]. De plus, en présence de modèles massivement complexes, les solveurs sous-jacents (comme OR-Tools ou Gecode) peuvent être arrêtés par un "timeout". Dans ce cas, la *complétude* formelle demeure, mais la solution optimale ne peut être atteinte. La bisimulation est donc une propriété intrinsèque à l'espace de recherche, et non une garantie sur le temps de calcul des solveurs.

## 11 Conclusion

L'adaptation du concept de bisimulation à notre langage déclaratif permet d'établir une fondation formelle solide pour notre projet de recherche. En démontrant formellement la correspondance entre les entités du DSL et leurs variables MiniZinc associées via une traduction  $\mathcal{T}$  et un mapping *decode*, nous certifions que le problème résolu est le reflet exact du besoin métier. Cette démarche garantit qu'il n'y a ni perte d'information, ni introduction d'anomalies lors de la résolution, consolidant ainsi la valeur académique et technique de la plateforme.

## Références

- [1] Gordon D. Plotkin. *A Structural Approach to Operational Semantics*. DAIMI FN-19, Aarhus University, 1981.

- [2] Robin Milner. *Communication and Concurrency*. Prentice-Hall, 1989.
- [3] Davide Sangiorgi. *Introduction to Bisimulation and Coinduction*. Cambridge University Press, 2011.
- [4] Xavier Leroy. *Formal Verification of a Realistic Compiler*. Communications of the ACM, 52(7) :107–115, 2009.
- [5] Klaus Schneider, Gert Smolka, et Michel Hack. *An Inductive Proof Method for Simulation-based Compiler Correctness*. Proceedings of Formal Methods, 2006.
- [6] Nicholas Nethercote, Peter J. Stuckey, Ralph Becket, Sebastian Brand, Gregory J. Duck, et Guido Tack. *MiniZinc : Towards a Standard CP Modelling Language*. Constraint Programming (CP), 2007. <https://www.minizinc.org>.